PEER AUDIT REPORT

Chainvoice

Smart Contracts · Frontend · Backend · Deployment

Project	Chainvoice: a decentralized invoicing dApp for EVM chains (create, send and pay invoices in native coin or ERC-20, with invoice details encrypted end to end)
Repositories	github.com/StabilityNexus/Chainvoice (contracts and frontend live in one repo)
Audited commit(s)	Chainvoice @ `a36a29b` (`main`, merge of PR #328, 24 Oct 2026)
Live deployment	https://chainvoice.stability.nexus/
Audit period	24 Oct 2026 to 24 Oct 2026
Auditor(s)	Rohan Shinde, @rohans02
Project maintainer(s)	Atharva Naik, @Atharva0506
Mentor / reviewer	Karan (@kumawatkaran523)
Report version	v1.0 · Final

Prepared under the Stability Nexus / AOSSIE peer audit programme.

Document Control

Revision History

Version	Date	Author	Changes
v0.1	24 Oct 2026	Rohan Shinde	Initial draft shared with maintainers (code review, automated analysis, functional testing)
v1.0	25 Oct 2026	Atharva Naik	Final: all 5 findings fixed (PRs #332, #335). Minor observations are tracked by the maintainers for after launch.

Distribution

Name	Role	Contact
Atharva Naik	Project maintainer	GitHub @Atharva0506 / Discord @atharva0506
Karan	Mentor	GitHub @kumawatkaran523 / Discord @karankk9616

Disclaimer

This report is a peer review performed by a fellow contributor within a limited time and against a fixed version of the code listed in the Scope section. It is not a guarantee that the software is free of defects or vulnerabilities, and it is not financial or legal advice. Findings apply only to the audited commits; later changes are not covered unless a re-audit is recorded in the Revision History.

Section 8 (Verified-Absent Checklist) records issue classes that the auditor actively checked and did not find. Each entry states how it was verified so readers can judge the strength of the claim. An unchecked or N/A entry makes no claim either way.

Contents

1.Executive Summary (1.1 Overview, 1.2 Findings at a Glance, 1.3 Overall Assessment, 1.4 Key Recommendations)
2.Audit Scope (2.1 Components in Scope, 2.2 Deployed Contracts, 2.3 Out of Scope)
3.Methodology (3.1 Approach, 3.2 Tools, 3.3 Test Environment)
4.System Overview and Trust Assumptions (4.1 Architecture, 4.2 Privileged Roles, 4.3 Trust Assumptions)
5.Severity Classification (5.1 Risk Matrix, 5.2 Severity Definitions, 5.3 Finding IDs)
6.Findings Summary
7.Detailed Findings
8.Verified-Absent Checklist (8.1 Smart Contracts, 8.2 Frontend: Functional, 8.3 Frontend: Security, 8.4 Backend / Server, 8.5 Deployment and Repository)
9.Functional Test Results (9.1 User Flows by Network, 9.2 Pages, Buttons and Links, 9.3 Automated Scores)
10.Recommendations and Conclusion (10.1 General Recommendations, 10.2 Conclusion, 10.3 Sign-off)

- Appendix A. Tool Output
- Appendix B. Glossary

1. Executive Summary

1.1 Overview

Chainvoice lets an issuer create an invoice on-chain that commits only to `keccak256` of the invoice details, and send the details to the payer ECIES-encrypted through a ThruBox relay. The payer verifies the details against the on-chain hash and pays in native coin or an ERC-20 token, plus a flat native fee per invoice. The audit covered the Solidity contract (`contracts/src/Chainvoice.sol`), its deploy script, the React/Vite frontend, the CI workflows and the live deployments at commit `a36a29b`. The relay server is a separate project and was not audited.

1.2 Findings at a Glance

Critical	High	Medium	Low	Informational
0	0	2	1	2

Status	Count	Meaning
Open	0	Not yet addressed by the maintainers
Acknowledged	0	Accepted as a risk or deferred, with a stated reason
Fixed	5	Resolved and the fix verified by the auditor (link the PR)
Won't fix	0	Maintainers disagree; reason recorded in the finding

1.3 Overall Assessment

	Verdict	When to use
☑	Ready	No open Critical or High findings; checklist complete.
☐	Ready after fixes	Open High/Medium findings with clear fixes; no Critical.
☐	Not ready	At least one open Critical finding, or core flows broken.

The contract is small, well tested (122 passing tests, 98.9% line coverage of Chainvoice.sol at the audited commit) and I found no way to lose or lock funds: payments use exact msg.value, checks-effects-interactions plus a reentrancy guard, and a balance-delta check against fee-on-transfer tokens. The frontend is careful too: amounts and tokens always come from the chain, never from the off-chain payload. This report records five findings: two Medium (a hostile-sender payload that crashed the Received page, CV-FE-01, and setup docs pointing at a retired Sepolia contract, CV-OPS-01), one Low (amount fields changing on mouse-wheel scroll, CV-FE-09) and two Informational (no event from withdrawFees, CV-SC-01, and contradictory licence signals, CV-OPS-03). All five are fixed (PRs #332 and #335). Other observations from testing are minor UI, accessibility and hardening items with no risk to funds, which the maintainers will address after launch. The audit is complete and the project is ready for its mainnet launch.

1.4 Key Recommendations

1. Validate the fields of batchInfo and render it defensively (CV-FE-01). Fixed in PR #335.
2. Point README.md and frontend/.env.example at the current deployments (CV-OPS-01). Fixed in PR #335.
3. Stop amount fields from changing on scroll (CV-FE-09), emit an event from withdrawFees (CV-SC-01) and make the licence signals consistent (CV-OPS-03). Fixed in PRs #335 and #332.

2. Audit Scope

2.1 Components in Scope

Component	Repository / Location	Commit / Version	Notes
Smart contracts	github.com/StabilityNexus/Chainvoice `contracts/`	`a36a29b`	`src/Chainvoice.sol` (539 lines), `script/Chainvoice.s.sol`; tests read for coverage only
Frontend	github.com/StabilityNexus/Chainvoice `frontend/`	`a36a29b`	`src/**` (about 25k lines incl. ABI and country list). Live URL: https://chainvoice.stability.nexus/ (bundle `index-CITqXL9g.js`, deployed 24 Oct 2026 06:02 GMT)
Backend / server	none in this repo	n/a	Invoice transport is the external ThruBox relay (AOSSIE-Org/ThruBox-Server); see 2.3
Other	`.github/workflows/*`, `Deployments.md`, `README.md`, `docker-compose.yml`	`a36a29b`	CI, deployment records, setup docs

2.2 Deployed Contracts

Addresses from `Deployments.md`, cross-checked against the live frontend bundle. "Bytecode = audited" means the runtime bytecode read with `cast code` equals `forge inspect Chainvoice deployedBytecode` at `a36a29b` after stripping the CBOR metadata tail.

Network	Chain ID	Contract	Address	Source verified?
Ethereum Sepolia (used by live site)	11155111	Chainvoice	`0xff10Cda545E10656C2DcBEAB6C9eC375E136363a`	Partial only (Blockscout "partially verified"; Etherscan and Sourcify: not verified). Bytecode = audited: yes
Ethereum Sepolia (PR previews)	11155111	Chainvoice	`0x094642FEbF43d17C44dbe07b74AE3A18fEdC7390`	No (Blockscout, Etherscan, Sourcify)
Mordor (ETC testnet)	63	Chainvoice	`0xeF6669DDEd0078A8Cfdd758A40D194b1cC2120d2`	Yes (Sourcify exact match; Blockscout fully verified). Bytecode = audited: yes
Polygon Amoy	80002	Chainvoice	`0xeF6669DDEd0078A8Cfdd758A40D194b1cC2120d2`	Yes (Sourcify exact match). Bytecode = audited: yes
Ethereum Sepolia (retired)	11155111	Chainvoice (older ABI)	`0x65eb0ca96f972c5a0cdaa623a5b54650e499df5b`	No. Older ABI (`owner`, `fee`); see CV-OPS-01
Ethereum Classic / Polygon (v1, legacy)	61 / 137	Chainvoice v1	`0xD044A85a5daC307217B9bF313A90E8a60AF7DdCe`	Not on Sourcify. Out of scope: v1 code, not enabled in the frontend

2.3 Out of Scope

- Third-party libraries used as-is: OpenZeppelin `IERC20`/`SafeERC20`, forge-std, wagmi, viem, ethers, eciesjs, `@stability-nexus/walletlink`, `@aossie-org/thrubox-client`, `@aossie-org/idb-backup`, jsPDF, fflate.
- External services: the ThruBox relay (`thrubox-server.onrender.com`, `thrubox-dev-server.onrender.com`), RPC providers, wallets, block explorers, Cloudflare Pages previews.
- The reusable workflow `StabilityNexus/ContributorAutomation` called from `track-contributor-prs.yml`.
- Legacy v1 deployments on Ethereum Classic and Polygon mainnet (the repo marks them unsupported in `SECURITY.md`).

- Treasury actions that only the `dev` key can perform (set fee, set receiver), and `withdrawFees` from the UI. They are covered by the contract tests.

3. Methodology

3.1 Approach

1.Read the documentation and README, and run the project locally from a clean clone.
2.Map the system: actors, privileged roles, user flows and trust assumptions (Section 4).
3.Manual line-by-line review of in-scope code.
4.Automated analysis with the tools in 3.2.
5.Functional testing of every page, button, link and flow on each supported network (Section 9).
6.Work through the Verified-Absent Checklist (Section 8).
7.Share the draft with maintainers, then verify fixes and record their responses.
Notes: all seven steps are done for this draft except the remediation part of step 7. The contract was reviewed line by line by the auditor. The frontend payment, relay, crypto, chain-config and key-derivation code was reviewed line by line by the auditor; the share, import, backup and create code was first swept with an AI-assisted review, and every frontend finding was then re-checked by the auditor in the code and with the PoC scripts in `tool-output/poc/`. Functional testing (step 5) was driven with a browser-automation agent under the auditor's supervision; it never interacted with the wallet, and every wallet request was approved or rejected by a human. Each failure it reported was re-checked against the code before going into this report; two reported failures did not hold up (dashboard drawer not keyboard reachable: the code uses focusable MUI `ListItemButton`s; batch invoices not grouped on Sent: that page has no batch grouping by design) and are not reported. Flows that need the payer's wallet were run and confirmed by the project team and are marked as such in Section 9.

3.2 Tools

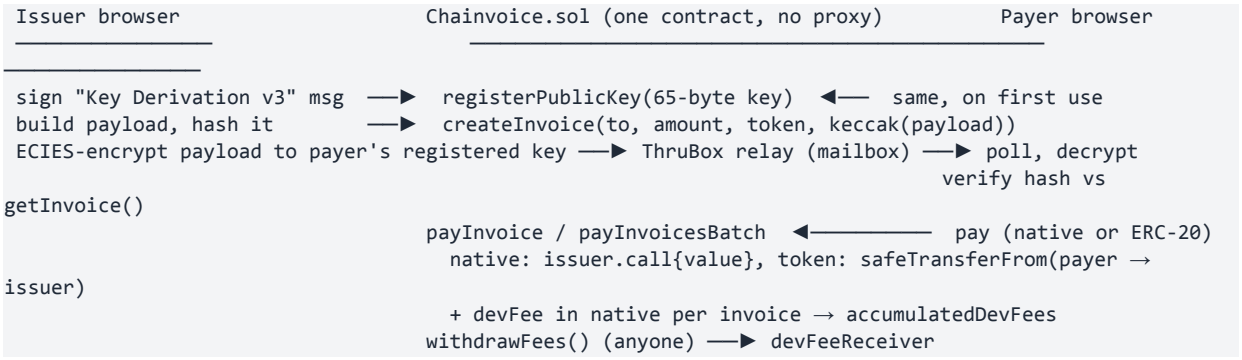
Tool	Version	Purpose	Result summary
Foundry (forge test / coverage)	forge 1.8.3 (cae51ad), solc 0.8.19	Contract tests and coverage	122 passed, 0 failed. `Chainvoice.sol`: 98.93% lines, 93.10% statements, 70.49% branches, 100% functions
Slither	0.11.6 (run in an isolated `uvx` env, same version as CI)	Static analysis	11 results: 1 High-impact detector (`arbitrary-send-eth`) and 1 Medium (`reentrancy-balance`), both triaged as false positives; rest informational
npm audit	npm 10.9.0, Node 22.12.0	Dependency vulnerabilities (`--omit=dev`)	31 (13 high, 18 moderate, 0 critical), all transitive
Lighthouse	12.8.2 (default mobile profile, headless Chrome)	Performance, accessibility, SEO	`/`: 35 / 100 / 100 / 100. `/#/dashboard/create`: 46 / 100 / 100 / 100. `/#/about`: 48 / 100 / 100 / 100
axe-core	4.10.2 (injected into the live page)	Accessibility	`/`: 1 critical, 1 serious. `/#/dashboard/create`: 3 critical, 3 serious. `/#/about`: 1 critical, 31 serious. (Lighthouse's smaller axe subset scored 100)
Other	Jest 30 (`npm run test:ci`), ESLint 9 (`npm run lint`), Vite 6 (`npm run build`), cast 1.8.3, curl	Frontend tests, lint, build; on-chain reads; headers	27 suites / 551 tests passed, 97.57% statements; lint exit 0; build exit 0

3.3 Test Environment

Browsers	Chrome (desktop) driven by a browser-automation agent; headless Chrome for Lighthouse
Wallets	MetaMask via WalletLink; every wallet request approved or rejected by a human
Networks	Sepolia (full set), Ethereum Classic Mordor and Polygon Amoy (smoke set), plus an unsupported chain for the wrong-network check. Read-only RPC checks on all three testnets
Devices / viewports	About 767 px (tablet/mobile layout), 375 px and 1440 x 900 px (simulated with same-origin frames)
Operating systems	Windows 11 (analysis host)

4. System Overview and Trust Assumptions

4.1 Architecture



- **Contract.** `Chainvoice.sol` stores `InvoiceDetails` (id, from, to, amountDue, tokenAddress, isPaid, isCancelled, invoiceDataHash), per-user sent/received id lists, and a public-key registry. Tokens move straight from payer to issuer; the contract only holds native dev fees. Fee settings are controlled by `dev` (Section 4.2).
- **Frontend.** React 18 + Vite, HashRouter, wagmi/WalletLink for wallets, ethers v6 for contract calls. A chain is "supported" exactly when `VITE_CONTRACT_ADDRESS_<chainId>` is set at build time (`frontend/src/utils/chainConfig.js#L76-L79`). Invoice payloads live only in the browser's IndexedDB; the on-chain record is the authority for parties, amount and token (`frontend/src/utils/chainInvoice.js#L125-L169`). Messaging keys are derived from a wallet signature over a message that names the site origin (`frontend/src/services/relay/relayKeyManager.js#L88-L103`).
- **Server.** No server in this repo. The ThruBox relay is a world-readable, world-writable mailbox of opaque ciphertexts.
- **Hosting.** GitHub Pages from the `gh-pages` branch (`deploy.yml`). PR previews on Cloudflare Pages with their own Sepolia contract and relay.

4.2 Privileged Roles

All 16 external/public state-changing or view functions were enumerated (`grep -n "function " src/Chainvoice.sol`). Only four are gated (`onlyDev`), one by `pendingDev`; the rest are open to anyone.

Role	Held by	Powers	Risk if compromised
dev	`0x69fF…B809` on all current deployments. On Sepolia this EOA has an EIP-7702 delegation (`0xef0100` + `0x63c0…e32b`, a MetaMask smart-account delegator); on Mordor and Amoy it is a plain EOA. README advises a multisig	`setDevFee` (any value, no cap, including 0), `setDevFeeReceiver` (any non-zero address, one step), `proposeDev` / `cancelDevProposal` (two-step handover)	Attacker can set `devFee` very high so every payment needs that fee (payments with the old fee revert, nothing is overcharged because `msg.value` must match exactly), or set it to `type(uint256).max` so `payInvoicesBatch` overflows and every payment reverts. Can redirect all future fee withdrawals. Cannot touch invoice funds or ERC-20 allowances
pending Dev	Zero on all deployments (read with `cast call pendingDev()`)	`acceptDev()` to become `dev`	Only after the current dev proposes it
devFeeR eceiver	`0x6F74…4C59` (Sepolia current, Mordor, Amoy; also 7702-delegated on Sepolia), `0x69fF…B809` on the preview contract	Receives `withdrawFees()` payouts; no other power	Loses only accrued native fees

Role	Held by	Powers	Risk if compromised
Invoice issuer (`from`)	Any address	`cancelInvoice` on own unpaid invoices	n/a (per-invoice)
Invoice payer (`to`)	Any address	Only `to` may pay an invoice	n/a
Anyone	Any address	`createInvoice(sBatch)` to any recipient, `registerPublicKey` for self, `withdrawFees` (always pays `devFeeReceiver`)	n/a
Repo / CI	GitHub maintainers; `CLOUDFLARE_API_TOKEN`, `GIST_PAT` secrets	Publish the production site (push to `main`), publish previews, label-gate fork previews	A malicious build on `main` could change contract addresses or key-derivation code for every visitor

4.3 Trust Assumptions

- **dev fee is set at payment time, without a cap.** `payInvoice` and `payInvoicesBatch` read `devFee` when the payer pays, not when the invoice was created, and `setDevFee` has no upper bound. Both are deliberate (tests `testSetDevFee_NoCap`, `testSetDevFee_DevCanSetZero`, `testPayInvoicesBatch_Native_ChargesInitialThenNewFee`). Payers are protected from overpaying by the exact `msg.value` checks, but the dev key can stop all payments by raising the fee. Users trust the dev key not to do that.
- **The fee is always charged in native coin**, also for ERC-20 invoices (`Chainvoice.sol#L307`). Payers need native coin even when paying in a token. Deliberate (`FeeMustBeNative` error, `getPaymentStatus` comment).
- **Anyone can invoice anyone.** There is no allow-list of senders or tokens. A payer must check the issuer address before paying; the display name in a payload is whatever the issuer typed. `_validateToken` only checks that `balanceOf`/`allowance` answer, so an issuer can name any contract as the token (this only hurts the issuer).
- **The relay is untrusted for confidentiality and integrity** (payloads are encrypted and hash-checked) but trusted for availability: if the relay drops a message, the payer only sees the on-chain summary.
- **Key safety depends on the user not signing the derivation message on another site.** The message names the origin and warns about this (`relayKeyManager.js#L88-L103`). Wallets that produce non-deterministic signatures would derive a different key each time; this was not tested.
- **The build pipeline is trusted.** Contract addresses and relay URL are baked into the bundle at build time from repository variables (`deploy.yml`).
- **RPC providers return honest chain data.** All verification (hash, parties, amount, decimals) runs in the browser against the configured RPC.

5. Severity Classification

Each finding is rated by combining its impact (what happens if it occurs) with its likelihood (how easily it can occur).

5.1 Risk Matrix

Impact ↓ / Likelihood →	High	Medium	Low
High	**Critical**	**High**	**Medium**
Medium	**High**	**Medium**	**Low**
Low	**Medium**	**Low**	**Low**

5.2 Severity Definitions

Severity	Definition and examples
Critical	Direct loss or permanent lock of user funds, or full takeover of the system, exploitable by anyone. Example: anyone can withdraw another user's payment.
High	Funds at risk under realistic conditions, or a core flow is broken for all users. Example: invoice creation always reverts on a supported network.
Medium	Limited loss, or a core feature broken for some users or inputs; a workaround exists. Example: wrong token decimals shown for a 6-decimal token.
Low	Minor impact, unlikely conditions, or degraded UX without functional loss. Example: missing loading state after a transaction is sent.
Informational	No direct impact: code quality, gas, documentation, best practice. Example: unused import, missing NatSpec.

5.3 Finding IDs

Findings use the format PROJECT-AREA-NN, for example FATE-SC-01. This report uses the project code **CV** (Chainvoice).

Area code	Covers
SC	Smart contracts
FE	Frontend (UI, functional, client-side security)
BE	Backend / server
OPS	Deployment, configuration, CI, repository hygiene

6. Findings Summary

ID	Title	Component	Severity	Status
CV-FE-01	Hostile `batchInfo` in a hash-valid payload crashes the Received page on every visit	Frontend	Medium	Fixed
CV-OPS-01	README and `.env.example` point to a retired Sepolia contract the current app cannot use	Ops	Medium	Fixed
CV-FE-09	Amount inputs change silently on mouse-wheel scroll	Frontend	Low	Fixed
CV-SC-01	`withdrawFees()` emits no event	Contracts	Informational	Fixed
CV-OPS-03	Contradictory licence signals (GPL-3.0 repo, `Unlicense` contract, `UNLICENSED` script)	Ops	Informational	Fixed

7. Detailed Findings

CV-FE-01
Hostile `batchInfo` in a hash-valid payload crashes the Received page on every visit

Severity	Medium (Impact: Medium · Likelihood: Medium)
Component	Frontend
Location	`frontend/src/utils/invoicePayloadShape.js#L80-L82`, `frontend/src/page/ReceivedInvoice.jsx#L1786-L1790`, `frontend/src/page/ReceivedInvoice.jsx#L1009-L1054`
Network(s)	All
Status	Fixed

Description

The shape check accepts any flat object as `batchInfo`:

```
// invoicePayloadShape.js#L80-L82
if (!isAbsent(payload.batchInfo) && !isFlatRecord(payload.batchInfo, [])) {
  return "batchInfo";
}
```

The Received table then assumes `batchId` is a string:

```
// ReceivedInvoice.jsx#L1786-L1790
{invoice.batchInfo && (
  ...
  label={`Batch #${invoice.batchInfo.batchId.slice(
    -4
  )} (${invoice.batchInfo.index + 1}/${invoice.batchInfo.batchSize
```

An issuer can commit on-chain to a payload with `batchInfo: {}` (or `batchId: 12345`). That payload passes the hash check, the shape check and the field match, so `buildChainInvoice` marks it trusted. `storeRelayInvoice` (`ReceivedInvoice.jsx#L1009-L1054`) stores it in the payer's IndexedDB with no click from the payer. Rendering the row then throws on `.slice`.

Impact

`PageErrorBoundary` (`Home.jsx#L363-L376`) catches the error, so the whole Received section shows the error fallback every time that invoice is on the current page. The payer cannot pay or export the other invoices on that page from the UI. There is no in-app way to delete the stored record; clearing site data also deletes the user's other local invoice details. Any sender can do this to any address that has registered a messaging key (keys are public on-chain), for the cost of one `createInvoice` transaction. No funds are at risk.

Steps to Reproduce / Proof of Concept

1. Run `node tool-output/poc/fe-payload-poc-1.mjs` from `Chainvoice/frontend` (imports the real modules at `a36a29b`).
2. The script builds `{amountDue:'5', …, items:[], batchInfo:{}}`, hashes it, and checks it the way the page does.
3. Observed: `shape error: null`, `evaluate: ok`, `trusted? true batchInfo {}`, then `render expr throws: Cannot read properties of undefined (reading 'slice')`. With `batchId: 12345`: `batchId.slice is not a function`, still trusted. Expected: payload rejected as "invoice format".

Evidence: `tool-output/poc/fe-payload-poc-1-output.txt`.

Recommendation

Validate the fields the UI reads, and render defensively:

```
// invoicePayloadShape.js
const isBatchInfo = (b) =>
  isPlainObject(b) &&
  typeof b.batchId === "string" &&
```

```
  Number.isInteger(b.index) && b.index >= 0 &&
  Number.isInteger(b.batchSize) && b.batchSize > b.index;
if (!isAbsent(payload.batchInfo) && !isBatchInfo(payload.batchInfo)) return "batchInfo";

label={`Batch #${String(invoice.batchInfo.batchId ?? "").slice(-4)} …`}
```

A per-row error boundary would also stop one bad row from hiding the whole table.

Maintainer Response

Accepted and fixed in PR #335, commit 7aaa6035 (merged as a44ba2c1): batchInfo must have a non-empty string batchId, an integer batchSize ≥ 1 and an integer index with 0 ≤ index < batchSize, and the batch label renders defensively. buildChainInvoice re-runs the check on every render, so records already stored in IndexedDB fall back to the on-chain-only view. Both PoC payloads are regression tests.

Auditor Verification

Fix merged to main at a44ba2c1 (PR #335).

CV-OPS-01
README and `.env.example` point to a retired Sepolia contract the current app cannot use

Severity	Medium (Impact: Low · Likelihood: High)
Component	Ops
Location	`README.md#L196-L201`, `README.md#L238-L243`, `frontend/.env.example#L3-L9`
Network(s)	Sepolia
Status	Fixed

Description

The README's "Environment Variables" and "Deployed Contracts / Current" sections, and `frontend/.env.example`, all give `VITE_CONTRACT_ADDRESS_11155111=0x65eb0ca96f972c5a0cdaa623a5b54650e499df5b` and describe it as matching the current contract. `Deployments.md` says that address was retired on 2026-10-19 and replaced by `0xff10…363a`, which is what the live site uses. The README setup step 5 tells contributors to `cp frontend/.env.example frontend/.env`, and `docker-compose.yml` reads the same file.

The retired contract has the older fee ABI. Read-only calls with the current frontend's ABI:

```
getReceivedInvoices(address,uint256,uint256) -> execution reverted
getSentInvoices(address,uint256,uint256)     -> execution reverted
devFee()                                      -> execution reverted
fee() = 500000000000000, owner() = 0x69fF…B809
```

`.env.example` also leaves Mordor (63) and Amoy (80002) blank although both have current deployments.

Impact

Every contributor who follows the README gets a local app whose Sent and Received pages show "Unable to load invoices" and whose pay buttons fail on `devFee()`. Keys they register land on a contract production does not read. No user funds or production users are affected; the cost is lost contributor time and confusing bug reports.

Steps to Reproduce / Proof of Concept

1.Fresh clone, `cd frontend && cp .env.example .env && npm ci && npm run dev`.

2.Connect a wallet on Sepolia and open `/#/dashboard/sent`.

3.Observed: the error "Unable to load invoices…", because `getSentInvoices(address,uint256,uint256)` reverts on `0x65eb…`. Expected: the list loads.

Evidence: `tool-output/onchain.txt` (cast calls against both addresses). The revert is confirmed on-chain.

Recommendation

Replace the address in both files with `0xff10Cda545E10656C2DcBEAB6C9eC375E136363a`, add `VITE_CONTRACT_ADDRESS_63` and `VITE_CONTRACT_ADDRESS_80002` with `0xeF6669DDEd0078A8Cfdd758A40D194b1cC2120d2`, and move `0x65eb…` to the "superseded" list in the README. Keeping a single source (`Deployments.md`) and linking to it from the README would stop the copies drifting again.

Maintainer Response

Accepted and fixed in PR #335, commit 7aaa6035 (merged as a44ba2c1): README and frontend/.env.example give the current Sepolia, Mordor and Amoy addresses, list 0x65eb…df5b as retired, and point to Deployments.md as the single record of deployments.

Auditor Verification

Fix merged to main at a44ba2c1 (PR #335).

CV-FE-09
Amount inputs change silently on mouse-wheel scroll

Severity	Low (Impact: Low · Likelihood: Medium)
Component	Frontend
Location	`frontend/src/page/CreateInvoice.jsx#L1488-L1496` (Qty), `#L1506-L1514` (Unit Price), other `type="number"` inputs at `#L1541`, `#L1574`, `#L1647`, `#L1662`, `#L1677`, `#L1694`; `frontend/src/page/CreateInvoicesBatch.jsx#L1439`, `#L1459`
Network(s)	All
Status	Fixed

Description

Quantity, price, discount and tax fields are plain `<Input type="number" step="any">` with no wheel handling (`grep -rn "onWheel" frontend/src` → 0 hits):

```
// CreateInvoice.jsx#L1506-L1514
<Input
  type="number"
  placeholder="0"
  ...
  name="unitPrice"
  min="0"
  step="any"
```

In Chrome, scrolling the mouse wheel over a focused number input steps its value. The total updates, but nothing draws attention to the change.

Impact

An issuer who scrolls down to the submit button with the cursor over a focused amount field can submit an invoice for a different amount than they typed. The wallet shows `createInvoice` with a raw base-unit amount, which is easy to miss. The issuer can cancel the invoice, and the payer sees the amount before paying, so no funds are lost directly.

Steps to Reproduce / Proof of Concept

1. On Sepolia, open `/#/dashboard/create`, enter client `0x000000000000000000000000000000000000dEaD`, one item, Unit Price `0`.
2. Leave the cursor over the Unit Price field and scroll the page down to the submit button with the mouse wheel.
3. Observed: Unit Price became `18` with no visible cue, and the form passed validation as an 18 ETH invoice. Expected: the value stays `0` and validation rejects it.

Evidence: that form was later submitted and is on-chain as invoice #2 on Sepolia `0xff10…363a`: `amountDue = 18000000000000000000` (18 ETH) to `0x…dEaD`, where 0 was typed (`cast call getInvoice(2)`,

`tool-output/onchain-test-invoices.txt`). A clean re-run without the wheel shows a zero price is correctly blocked.

Recommendation

Blur number inputs on wheel, or switch to text inputs with `inputMode="decimal"`, which also avoids float parsing of amounts:

```
<Input type="number" onWheel={(e) => e.currentTarget.blur()} ... />
```

Showing the final total in a confirm step before the transaction would also help.

Maintainer Response

Accepted and fixed in PR #335, commit 7aaa6035 (merged as a44ba2c1): the shared Input component blurs number inputs on wheel, covering all 13 amount, quantity, discount and tax fields. Checked in the browser: a typed value stays unchanged after scrolling over the focused field.

Auditor Verification

Fix merged to main at a44ba2c1 (PR #335).

CV-SC-01
`withdrawFees()` emits no event

Severity	Informational (Impact: none on funds · Likelihood: n/a)
Component	Smart contracts
Location	`contracts/src/Chainvoice.sol#L528-L538`
Network(s)	All
Status	Fixed

Description

```
function withdrawFees() external {
    if (accumulatedDevFees == 0) revert NoFeesAvailable();
    uint256 amount = accumulatedDevFees;
    accumulatedDevFees = 0;
    (bool success, ) = payable(devFeeReceiver).call{value: amount}("");
    if (!success) revert WithdrawFailed();
}
```

Every other state change in the contract emits an event (17 `emit` statements), but this one, which moves the whole fee balance and can be called by anyone, does not.

Impact

Indexers, the Treasury page and anyone auditing fee flows cannot see when, by whom or how much was withdrawn without tracing internal transactions.

Steps to Reproduce / Proof of Concept

1. Copy `tool-output/poc/AuditPoC.t.sol` into `contracts/test/` and run `forge test --match-contract AuditPoCTest -vvv`.
2. Observed: `[PASS] test_PoC_withdrawFees_emitsNoEvent()`: receiver is paid 0.0005 ether and `vm.getRecordedLogs()` is empty.

Evidence: `tool-output/poc/forge-poc-output.txt`.

Recommendation

```
event DevFeesWithdrawn(address indexed caller, address indexed receiver, uint256 amount);
...
emit DevFeesWithdrawn(msg.sender, devFeeReceiver, amount);
```

Maintainer Response

Accepted and fixed in PR #332, commit f4d9d2fe: withdrawFees() emits DevFeesWithdrawn(caller, devFeeReceiver, amount) before the transfer, with tests for a third-party caller and after a receiver change. PR #332 merges before the mainnet deploy, so the mainnet contract includes the event.

Auditor Verification

Fix in commit f4d9d2fe (PR #332).

CV-OPS-03
Contradictory licence signals (GPL-3.0 repo, `Unlicense` contract, `UNLICENSED` script)

Severity	Informational (Impact: none on funds · Likelihood: n/a)
Component	Ops
Location	`LICENSE.md#L1-L4`, `contracts/src/Chainvoice.sol#L1`, `contracts/script/Chainvoice.s.sol#L1`, `contracts/test/Chainvoice.t.sol#L1`
Network(s)	n/a
Status	Fixed

Description

`LICENSE.md` is GPL-3.0 and `COPYRIGHT.md` says all works follow it. `Chainvoice.sol` declares `SPDX-License-Identifier: Unlicense` (a public-domain dedication), and the deploy script declares `UNLICENSED` (no licence granted). The contract's SPDX line is part of the verified source on explorers.

Impact

Readers of the verified contract see a different licence from the repository. No security effect.

Recommendation

Pick one and align the SPDX lines with `LICENSE.md` (for example `GPL-3.0-or-later`). Check with the mentors first if the org has a planned contracts licence.

Maintainer Response

Accepted and fixed in PR #332, commit f4d9d2fe, following Bruno’s decision: every Solidity file declares SPDX-License-Identifier: UNLICENSED with a comment linking to the repository for licensing, until the contract licence is published. The frontend stays GPL-3.0.

Auditor Verification

Fix in commit f4d9d2fe (PR #332).

8. Verified-Absent Checklist

Traditional audits list only the issues that were found. This section also records which classes of issues were checked and not found. Absence of evidence is not proof, so every ✅ must state how it was verified (tool output, test, or the exact review performed). Only mark ✅ when the check was complete for the whole scope.

Legend

✅	Verified absent. The auditor checked this and found no instance. Presumed not to exist in the audited version.
❌ ID	Found. Write the finding ID, e.g. ❌ PROJ-SC-02, and describe it in Section 7.
➖	Not applicable to this project (e.g. no backend). Say why in the last column.
⚠	Not checked, or only partly checked. Makes no claim either way.

8.1 Smart Contracts

#	Issue class checked	Result	How it was verified
SC-1	Reentrancy: external calls follow checks-effects-interactions or are guarded (nonReentrant)	✅	Listed all external calls: `.call{value}` ×3 (L304, L365, L536), `safeTransferFrom` + 2×`balanceOf` in `_pullToken` (L156-L163), `allowance` (L308, L373), `staticcall` ×2 in `_validateToken`. `payInvoice`/`payInvoicesBatch` set `isPaid` and fees before any call and are `nonReentrant`. `withdrawFees` zeroes the balance first. Re-entry into unguarded `createInvoice`/`cancelInvoice` during a payout cannot affect paid invoices (cancel requires `!isPaid`). Tests `*_IssuerReentryReverts` pass. Slither `reentrancy-balance` triaged as false positive (both callers `nonReentrant`)
SC-2	Missing access control on any state-changing external/public function	✅	Enumerated all 16 external functions (4.2). `setDevFee`, `setDevFeeReceiver`, `proposeDev`, `cancelDevProposal`: `onlyDev`. `acceptDev`: `msg.sender == pendingDev`. `cancelInvoice`: issuer only. `payInvoice(sBatch)`: `msg.sender == to` per invoice. `withdrawFees` is open but pays only `devFeeReceiver` (test `testWithdrawFees_AnyoneCanTriggerButReceiverIsPaid`)
SC-3	Unchecked ERC-20 transfer/transferFrom return values (no SafeERC20)	✅	Only token transfer is `token.safeTransferFrom` (L158, `using SafeERC20 for IERC20`); `grep -n "\.transfer(|transferFrom(" src/` → only the safe call. Test `testPayInvoicesBatch_NoReturnToken_Succeeds`
SC-4	Integer overflow/underflow (Solidity ≥ 0.8; every unchecked block reviewed)	✅	Pragma `0.8.19`; `grep -c unchecked src/Chainvoice.sol` → 0; `grep -c assembly` → 0. Possible overflow `devFee * n` (L340) reverts rather than wraps (see 4.3)
SC-5	Precision loss: division before multiplication, rounding in user's favour	✅	No division anywhere in `Chainvoice.sol` (`grep -n " / " src/Chainvoice.sol` → 0). Fee is a flat amount
SC-6	tx.origin used for authorisation	✅	`grep -c tx.origin src/Chainvoice.sol` → 0

#	Issue class checked	Result	How it was verified
SC-7	delegatecall to untrusted address, or selfdestruct	✅	`grep -c "delegatecall|selfdestruct"` → 0
SC-8	Unprotected initializer / uninitialised proxy	✅	No proxy, no `initialize` (`grep` → 0). All roles set in the constructor with zero checks (L93-L105); test `DeployChainvoice.t.sol`
SC-9	Unbounded loops over user-growable arrays (gas DoS)	✅	Loops are bounded by `MAX_BATCH = 50`, `MAX_CREATE_BATCH = 30` and `MAX_PAGE_LIMIT = 50` (L127-L134, checks L227, L328, L463). Per-user id arrays grow without bound but are only read through paginated views
SC-10	Stale, zero or negative oracle values accepted	➖	No oracle in the system
SC-11	Front-running / missing slippage or deadline protection on trades	✅	No price-dependent trades. Amounts are fixed per invoice and `msg.value` must equal `amountDue + devFee` exactly (L301, L307, L361, L371), so a fee change between quote and execution reverts instead of overcharging. Issuer cancelling in front of a payment only makes the payment revert
SC-12	Signature replay (missing nonce, chain ID or EIP-712 domain)	➖	The contract verifies no signatures (`grep -c "ecrecover|ECDSA"` → 0). The off-chain key-derivation signature is not an authorisation and is origin-bound (`relayKeyManager.js#L88-L103`)
SC-13	Native currency handling: wrong msg.value checks, locked ETH, failed refunds	✅	Exact-equality checks on every payable path; ERC-20 path requires `msg.value == devFee`. No `receive`/`fallback`, so plain ETH sends revert. Contract holds only `accumulatedDevFees`, withdrawable. Failed payout reverts the whole tx (`NativeTransferFailed`). Force-sent ETH (selfdestruct) would be stuck, but belongs to the sender who forced it
SC-14	Fee-on-transfer / rebasing / non-18-decimal tokens mishandled	✅	`_pullToken` checks the payee's balance delta and reverts with `TokenAmountShortfall` (L156-L163). Contract is decimal-agnostic (stores base units). Frontend reads decimals from the token contract, never from the payload (`chainInvoice.js#L67-L75`)
SC-15	Block timestamp or block number used as randomness	✅	`grep -c "block.timestamp|block.number|blockhash"` → 0
SC-16	State changes without events	✅ Fixed: CV-SC-01 (PR #332)	Checked every state-changing function for an `emit`; only `withdrawFees` lacks one
SC-17	Floating pragma or outdated compiler version	✅	Fixed `pragma solidity 0.8.19`, `evm_version = "london"` pinned deliberately for older EVM chains (`foundry.toml` comment). Slither's three 0.8.19 known bugs reviewed: `VerbatimInvalidDeduplication` (no `verbatim`), `FullInlinerNonExpressionSplitArgumentEvaluationOrder` (only custom optimizer sequences; none set), `MissingSideEffectsOnSelectorAccess` (only `IERC20.x.selector`, no side effects). None applies

#	Issue class checked	Result	How it was verified
SC-18	Unresolved Slither High/Medium detectors	✅	Slither 0.11.6: `arbitrary-send-eth` (withdrawFees always pays `devFeeReceiver`, set only by `dev`: false positive, already annotated in code) and `reentrancy-balance` (false positive, see SC-1). Annotations did not apply on my Windows CRLF checkout, see Appendix A. CI runs `slither --fail-high` on Linux
SC-19	Failing tests, or revert paths without tests	✅	122/122 pass. Coverage `Chainvoice.sol` 98.93% lines, 70.49% branches; every custom error has a revert test (`grep -c "expectRevert" test/Chainvoice.t.sol` read alongside the error list)
SC-20	Project: duplicate or mixed invoices in a batch	✅	Paid flag set in the validation pass so a repeated id hits `AlreadySettled` before funds move (L341-L356); `MixedTokenBatch` check (L352). Tests `RevertOnDuplicateId`, `RevertOnMixedTokens`
SC-21	Project: issuer or payer griefing another party's invoices	✅	Only `to` can pay and only `from` can cancel. An issuer whose address rejects ETH only makes their own invoices unpayable. A reverting issuer in a batch fails that batch only, and the frontend estimates gas first
SC-22	Project: spam invoices / unsolicited invoices	➖	Deliberate open design (anyone can invoice anyone). Listed as a trust assumption in 4.3; views are paginated so spam cannot block reads

8.2 Frontend: Functional

#	Issue class checked	Result	How it was verified
FE-1	Any route that crashes, shows a blank screen, or breaks on direct URL / refresh	✅ Fixed: CV-FE-01 (PR #335)	Code and PoC: a hostile payload crashes `/dashboard/pending` (CV-FE-01). Browser: all 10 main routes render after direct load and reload; unknown routes show the 404 page; path URLs redirect to hash routes (9.2)
FE-2	Any button that does nothing or the wrong thing	⚠ Minor, after launch	Every button in 9.2 exercised; `/feature` "Get Started" goes to 404 and `/working` is a leftover demo. All others behaved as labelled
FE-3	Broken internal or external links (404)	⚠ Minor, after launch	Internal: every `to=` / `navigate(` target compared with `App.jsx` routes; `/home` has no route, confirmed live. External: all footer, Terms and About links opened in new tabs and resolved
FE-4	Errors in the browser console during normal use	✅	Console read on every step of both browser passes: no errors from the app (only MetaMask extension warnings). Lighthouse best practices 100 on three pages
FE-5	Wallet connect, disconnect or account switch not reflected in the UI	✅	Connect: navbar items, network selector and account menu appear and Share hides. Account switch and disconnect: ✅ (project team)
FE-6	Wrong-network state not detected, or no prompt to switch	✅	Code: `SupportedNetworkGuard` wraps all `/dashboard/*` routes (`Home.jsx#L370-L372`) and `/treasure`, with a switch button per

#	Issue class checked	Result	How it was verified
			deployed chain. Browser: ✅ (project team)
FE-7	Rejected or failed transaction leaves the UI stuck	⚠ Minor, after launch	Payment handlers reset loading in `finally` (`ReceivedInvoice.jsx#L612-L617`, `#L788-L793`); reject-then-retry on payment ✅ (project team). The Import page can stay stuck on a malformed link
FE-8	No pending / confirmed / failed feedback for transactions	✅	Browser: progress and result toasts on create, batch create, cancel and resend; redirect after create. Payment feedback ✅ (project team)
FE-9	Forms accept empty, negative, too-large or malformed input	✅ Fixed: CV-FE-09 (PR #335)	Validation itself is sound: browser confirmed empty, own and invalid addresses and zero price are blocked with no wallet prompt; other cases covered by `invoiceValidation.test.js` (26 tests). But a mouse-wheel scroll can change an amount silently, and invoice #2 (18 ETH) reached the chain that way (CV-FE-09)
FE-10	Token amounts shown with wrong decimals or rounding	✅	Code: shown and paid amounts come from on-chain `amountDue` and token-contract `decimals` (`chainInvoice.js#L91-L169`). Browser: USDC read as 6 decimals; 2 × 1.5 shown as 3.0 USDC and stored on-chain as 3000000 (#4). The wrong token logo is a minor display issue, scheduled for after launch
FE-11	Layout broken at mobile width (375 px) or horizontal scrolling	⚠ Minor, after launch	No horizontal scroll at 375 px or 1440 px on the main pages; mobile menu works. `/about` heading clipped and wallet banner covers the Create header at 375 px
FE-12	Unreadable text in light or dark theme	⚠ Minor, after launch	App is dark-only. axe `color-contrast` flagged 3 elements on Create and 1 on About
FE-13	Accessibility: critical axe violations, no keyboard navigation	⚠ Minor, after launch	axe-core 4.10.2 on three pages: critical `button-name` and `label` violations. Keyboard: Create form fully reachable with visible focus
FE-14	Project: payload contradicts the on-chain record (amount, token, parties)	✅	Reviewed `chainInvoice.js`, `invoiceShareMatch.js`, `ReceivedInvoice.jsx#L885-L941`, `#L1009-L1054`: amount, token, decimals and parties always from chain; mismatching payloads shown as on-chain-only or blocked from payment (`isInvoicePayable`). PoC confirmed amount/decimals mismatch is caught
FE-15	Project: hostile but hash-valid payloads	✅ Fixed: CV-FE-01 (PR #335) · ⚠ minor items after launch	PoC scripts in `tool-output/poc/`

8.3 Frontend: Security

#	Issue class checked	Result	How it was verified
FS-1	Secrets, private keys or paid API keys shipped in the client bundle	✅	Regex scan of the live main bundle and all local `dist/assets/*.js` for 64-hex strings, AWS/GitHub/Stripe/Google key patterns, PEM headers and `PRIVATE_KEY`/`ETHERSCAN_API_KEY`: every 64-hex hit is a curve or library constant (secp256k1/ed25519 parameters, bytecode) (`tool-output/secret-scan.txt`). Live env values: three contract addresses, relay URL, empty `VITE_RELAY_API_KEY`. `git grep` for key patterns over the repo → 0 hits
FS-2	XSS: unsanitised user or on-chain data rendered as HTML	✅	`grep -rn "dangerouslySetInnerHTML|innerHTML|eval(|new Function|document.write" frontend/src` → 0. `ReactMarkdown` is used only for the bundled Terms text, without `rehype-raw`. No payload value is used as a URL or image `src`. PDF export uses jsPDF text; CSV escapes `= + - @`
FS-3	Contract addresses / ABIs in the frontend differ from the verified deployments	✅	Live bundle values (`VITE_CONTRACT_ADDRESS_11155111/63/80002`) equal the current rows of `Deployments.md`. `ChainvoiceABI.js` compared entry by entry with `forge inspect Chainvoice abi`: 68 = 68, no differences (`tool-output/abi-diff.txt`). On-chain bytecode = audited build (2.2)
FS-4	Missing security headers (CSP, X-Frame-Options / frame-ancestors)	⚠ Minor, after launch	`curl -sI` on the live site (`tool-output/live-site.txt`)
FS-5	External links opened without rel="noopener"	✅	15 `target="_blank"` in `Footer.jsx` (6), `About.jsx` (6), `TermsOfUseModal.jsx` (3); each has `rel="noopener noreferrer"` on the following line (15 matches)
FS-6	High or critical vulnerabilities in production dependencies	⚠ Minor, after launch	`npm audit --omit=dev` (`tool-output/npm-audit-omit-dev.txt`)
FS-7	Project: messaging key derivation and storage	✅	Message binds origin and address and warns the user (`relayKeyManager.js#L88-L103`); opaque origins rejected; session persistence is opt-in; caches cleared per address. PR previews use a separate contract so preview-origin keys cannot overwrite production keys (`Deployments.md`)

8.4 Backend / Server

The repository has no backend. Invoice transport is the external ThruBox relay, which is out of scope (2.3).

#	Issue class checked	Result	How it was verified
BE-1	Non-public endpoints reachable without authentication	➖	No server in this repo; relay is external
BE-2	Missing input validation on request bodies / parameters	➖	No server in this repo. Client-side validation of relay data is covered in FE-14/FE-15

#	Issue class checked	Result	How it was verified
BE-3	No rate limiting on expensive or abusable endpoints	➖	No server in this repo
BE-4	CORS allows unintended origins	➖	No server in this repo. Note: the preview relay accepts any origin by design (`Deployments.md`)
BE-5	Secrets committed to the repo or written to logs	✅	`git grep` for private-key, PEM, AWS, GitHub-token and Stripe patterns over the whole repo at `a36a29b` → 0 hits; only `.env.example` files are committed; `.env.preview` holds no credentials
BE-6	Error responses leak stack traces or internals	➖	No server in this repo
BE-7	Server keys can be drained or abused (e.g. relayer gas wallet)	➖	No relayer or server wallet; users pay their own gas

8.5 Deployment and Repository

#	Issue class checked	Result	How it was verified
OPS-1	Deployed contract source not verified, or not matching the audited commit	⚠ Minor, after launch	Sourcify v2 API, Blockscout API and Etherscan page for each address; runtime bytecode compared with the audited build for Sepolia, Mordor, Amoy (all equal minus metadata) (`tool-output/source-verification.txt`)
OPS-2	Privileged keys held by a single undocumented EOA	✅	`dev`/`devFeeReceiver` are documented in `Deployments.md` and read on-chain (4.2). They are single keys (EOA; 7702-delegated on Sepolia), on testnets only. README recommends a multisig; see 10.1 for mainnet
OPS-3	CI does not run build, lint and tests on pull requests	✅	`test.yml`: `npm ci`, lint, `test:ci`, build on `pull_request`, failing the job on any failure (L118-L128). `contracts.yml`: `forge build --sizes`, `forge test -vvv`, Slither `--fail-high` on `pull_request`. CodeQL on PRs
OPS-4	GitHub Actions not pinned to SHAs, or with excessive permissions	✅	Every `uses:` in 15 workflows is a full commit SHA. Workflow-level permissions are `{}`/read with per-job grants. `pull_request_target` jobs check out the default branch only (`pr-preview-approval.yml`) or no code. Minor: `foundry-toolchain` installs `version: stable` (unpinned binary)
OPS-5	README setup steps fail from a clean clone	✅ Fixed: CV-OPS-01 (PR #335)	Followed README on a clean clone: `forge build/test` and `npm ci/build/lint/test` succeed; the documented Sepolia address is the retired contract
OPS-6	Missing licence or security policy (SECURITY.md)	✅ Fixed: CV-OPS-03 (PR #332)	`LICENSE.md` (GPL-3.0) and `SECURITY.md` exist; SPDX headers contradict the licence. `SECURITY.md` notes private vulnerability reporting may not be enabled yet

9. Functional Test Results

Tested on the live site on 24 Oct 2026 against contract `0xff10…363a` (Sepolia) and `0xeF66…20d2` (Mordor, Amoy). Browser pages were driven with a browser-automation agent under the auditor's supervision; every wallet request was approved or rejected by a human. Expected behaviour comes from the code at `a36a29b`.

Result legend: ✅ pass · ❌ + finding ID · ➖ not applicable · ✅ (project team) = run and confirmed by the Chainvoice project team, not re-run by the auditor. · ⚠ = minor, scheduled after launch

Coverage taken from the project's own tests. Chainvoice has no browser or end-to-end tests and no tests of its page components, so flows that go through a page handler, the wallet, the live relay or a live chain were tested by hand. Where a manual step would only re-check logic that the project already unit-tests, the existing tests were relied on instead (all passing at `a36a29b`, Appendix A):

Not repeated by hand	Covered instead by
Form validation for zero, negative, too many decimals, missing fields, invalid email, on single and batch create	`frontend/tests/utils/invoiceValidation.test.js` (26 tests). In the browser, empty, own and invalid addresses and a zero price were confirmed blocked with no wallet prompt
Line-item totals	`invoiceCalculations.test.js` (9)
Token decimals handling	`tokenDecimalsVerification.test.js`, `nativeToken.test.js`, `chainInvoice.test.js`
Share-link edge cases (tampered, truncated, oversized, wrong chain, hash or field mismatch)	`invoiceShareCodec.test.js` (46), `invoiceShareVerify.test.js` (6)
Backup record validation edge cases	`invoiceBackup.test.js` (22)
Relay retries, rate limits, other-chain and undecryptable messages	`relayInvoiceMessaging.test.js` (27), `relayKeyManager.test.js` (39)
Filter, sort, pagination, PDF export edge cases	`invoiceFilterSortHelpers.test.js` (24), `invoicePagination.test.js` (11), `invoicePdfExport.test.js` (13)
Contract behaviour on every network	`contracts/test/Chainvoice.t.sol` (121). Mordor and Amoy run bytecode identical to the audited build (2.2), so they got a smoke test (register key, create native, pay native) instead of the full set

9.1 User Flows by Network

User flow	Sepolia	Mordor	Amoy	Unsupported chain
Connect wallet (WalletLink, EIP-6963)	✅	✅ (project team)	✅ (project team)	✅ (project team): switch prompt shown
Register / unlock messaging key	✅ (project team)	✅ (project team)	✅ (project team)	➖
Create single invoice, native coin	✅ (#3) · ✅ Fixed: CV-FE-09 (PR #335)	✅ (project team)	✅ (project team)	➖
Create single invoice, ERC-20 (USDC, 6 decimals)	✅ (#4; decimals read on-chain) · ⚠ minor (logo), after launch	➖ smoke only	➖ smoke only	➖
Create batch invoices	✅ (#5, #6 in one tx)	➖ smoke only	➖ smoke only	➖

User flow	Sepolia	Mordor	Amoy	Unsupported chain
Receive invoice via relay and see verified details	✅ (project team)	✅ (project team)	✅ (project team)	➖
Pay single invoice, native (incl. reject then retry)	✅ (project team)	✅ (project team)	✅ (project team)	➖
Pay single invoice, ERC-20 (approve then pay)	✅ (project team)	➖ smoke only	➖ smoke only	➖
Batch pay	✅ (project team)	➖ smoke only	➖ smoke only	➖
Cancel invoice (issuer side, then payer view)	✅ (#7) · payer view ✅ (project team)	➖ smoke only	➖ smoke only	➖
Resend invoice details	✅	➖ smoke only	➖ smoke only	➖
Share link / QR, then import	✅ link and QR generated · import ✅ (project team)	➖ smoke only	➖ smoke only	➖
Backup export / import (after clearing site data)	export ✅ · import ✅ (project team)	➖ smoke only	➖ smoke only	➖
View history: filter, sort, page size, drawer, CSV/PDF export	✅ (project team)	➖ smoke only	➖ smoke only	➖
Treasury: non-dev read-only view	✅ (controls disabled; Withdraw not clicked)	➖	➖	✅ (project team): switch prompt shown
Account switch and disconnect reflected without reload	✅ (project team)	➖	➖	➖

Test invoices created on Sepolia `0xff10…363a`: #2 (the CV-FE-09 case, 18 ETH to `0x…dEaD`), #3 native, #4 USDC, #5 and #6 batch, #7 cancelled (`tool-output/onchain-test-invoices.txt`).

9.2 Pages, Buttons and Links

Page / route	Element	Expected behaviour	Result
/	Terms of Use gate (first visit)	Blocks the page until accepted	✅
/	Navbar: Home / Dashboard / Treasury / About	Disconnected: only About (plus Share, Connect Wallet), by design (`Navbar.jsx#L181`). Connected: all four	✅
/	Navbar: Connect wallet	Opens the WalletLink picker	✅
/	Share button (only when disconnected)	Opens the AOSSIE share panel; hidden once connected	✅
/ (all pages)	Footer: Stability Nexus, GitHub, X, Discord, LinkedIn, Telegram	Each opens the matching URL in a new tab	✅
/ (all pages)	Footer: Terms of Use	Modal opens; links inside open in new tab; closes with button and Escape	✅
/	Mobile menu button	Opens and closes the menu	✅ · ⚠ minor (no accessible name), after launch

Page / route	Element	Expected behaviour	Result
`/dashboard`	(index)	Redirects to `/dashboard/create`	✅
`/dashboard/*`	Sidebar items (7), mobile drawer, desktop rail collapse/expand	Each opens its route; rail toggles	✅ (drawer at 767 px, rail at 1440 px)
`/dashboard/*`	Wrong network	Card with a switch button per deployed chain	✅ (project team)
`/dashboard/create`	Profile prompt / profile form	Saved locally, survives reload	✅
`/dashboard/create`	Custom token verify	Reads symbol and decimals on-chain	✅ (USDC, 6)
`/dashboard/create`	Submit with empty, own, invalid address, zero price	Blocked, no wallet prompt	✅
`/dashboard/create`	Amount fields	Value changes only when typed	✅ Fixed: CV-FE-09 (PR #335)
`/dashboard/create`	Submit valid	`createInvoice` tx, redirect to Sent	✅
`/dashboard/batch-invoice`	Two rows, submit	One `createInvoicesBatch` tx, both listed	✅
`/dashboard/sent`	List, cancel (confirm dialog), resend, share dialog, QR save	As labelled	✅
`/dashboard/sent`	Filter, sort, page size, drawer, CSV/PDF export	As labelled	✅ (project team)
`/dashboard/generate-link`	Share link / QR	Link and QR produced; warning that the link exposes details	✅
`/dashboard/pending`	Register key / unlock, pay, approve then pay, batch pay	As labelled	✅ (project team)
`/dashboard/import`	Paste link, verify, save	Verifies against chain, saves details	✅ (project team)
`/dashboard/settings`	Backup export	JSON file downloads	✅
`/dashboard/settings`	Backup import	Details restored, no duplicates	✅ (project team)
`/treasure`	Update Address, Update Fee, Withdraw	Disabled for non-dev; fee 0.0005 ETH and receiver shown	✅
`/about`	Contracts, Discord ×2, GitHub, contributor links, back-to-top	Open in new tab; 29/29 avatars load	✅ · ⚠ minor (placeholder address, unnamed links), after launch
`/feature`	"Get Started with Chainvoice"	Dashboard	⚠ minor (goes to #/home, 404), after launch
`/working`	"Open Drawer"	Should not exist	⚠ minor (leftover demo page), after launch
`/does-not-exist`	Not Found page, "Go Back Home"	Back to `/`	✅
`/dashboard/sent` (path URL, no `#`)	404.html redirect	Rewrites to `/#/dashboard/sent`	✅
Layout at 375 px and 1440 px	All main pages	No horizontal scroll, nothing cut off	✅ 1440 px · ⚠ minor at 375 px (clipped heading), after launch

9.3 Automated Scores

Lighthouse 12.8.2, default mobile profile, headless Chrome, 24 Oct 2026 (`tool-output/lighthouse-*.json`). Hash routes load the same document, so the three runs mostly measure the shared bundle (main chunk 941 KB, 310 KB gzip).

Page	Performance	Accessibility	Best practices	SEO
`/`	35	100	100	100
`/#/dashboard/create`	46	100	100	100
`/#/about`	48	100	100	100

10. Recommendations and Conclusion

10.1 General Recommendations

- **Treat every hash-valid payload as hostile input. The hash proves who wrote a payload, not that it is safe to render. Run one shared validator at every entry point (relay, share import, backup import) before storing (CV-FE-01, fixed). A fuzz test that feeds random JSON through getPayloadShapeError and then through the row renderers would catch this class.**
- **Keep one source of truth for addresses.** Generate the README snippet and `.env.example` from `Deployments.md`, or link to it, so they cannot drift (CV-OPS-01).
- **Before mainnet redeploys:** put `dev` and `devFeeReceiver` behind a multisig as the README already advises, verify every deployment with an exact match, and consider documenting a maximum fee the dev intends to use, since payers have no on-chain cap (4.3). A UI for `proposeDev`/`acceptDev` on the Treasury page would make the two-step handover easier to use safely.
- **Performance:** the main chunk is 941 KB. Lazy-loading heavy libraries only where used (jsPDF, html2canvas, jsQR, MUI on the dashboard) would likely lift the Lighthouse performance score from 35 to 48.
- **Branch coverage:** contract branch coverage is 70.49%. Adding tests for the remaining branches (for example `_pullToken`'s `afterBal < before` arm and `getPaymentStatus` token paths) is cheap.
- **Small UX polish seen in testing:** show the `dev` address on the Treasury page (it only says "Requires the dev address"), confirm "Copy link" and QR "Save" with a toast, and check why the first click on the Cancel icon in Sent sometimes does nothing.
- **Dependency hygiene:** move build-only packages to `devDependencies` and pin the Foundry toolchain version in CI (OPS-4 note).

10.2 Conclusion

Chainvoice is in good shape for a project of its size. The contract is minimal, owner-less apart from fee settings, fully tested and free of fund-loss paths that I could find; its deployed bytecode matches the audited code on all three current testnets. The frontend applies the right core rule (the chain decides amounts, tokens and parties) and is unusually careful about relay spoofing, preview isolation and key binding. All five findings in this report are fixed (PRs #332 and #335), and the core flows (create, receive, pay, cancel, share, backup) worked on Sepolia, Mordor and Amoy. The audit is complete and the project is ready for its mainnet launch; the remaining minor items can be handled afterwards.

10.3 Sign-off

Role	Name	Signature / GitHub handle	Date
Auditor	Rohan Shinde	@rohans02	[ ]
Project maintainer	Atharva Naik	@Atharva0506	[ ]
Mentor	Karan	@kumawatkaran523	[ ]

Appendix A. Tool Output

Raw output is in `./tool-output/`:

File	Content
`forge-build-test.txt`	`forge build` and `forge test` (122 passed)
`forge-coverage.txt`	`forge coverage --ir-minimum --report summary` (the project compiles with `via_ir`, so `--ir-minimum` was needed)
`slither.txt`	Slither 0.11.6, `slither . --config-file slither.config.json` (11 results). Slither was not installed on the host; it was run through `uvx` in an isolated cache, nothing installed system-wide. Note: the clone was checked out with `core.autocrlf=true` (CRLF), which shifts Slither's line mapping, so the in-code `slither-disable-next-line` comments did not suppress the two triaged detectors here. CI runs on Linux (LF)
`npm-ci.txt`, `npm-audit-omit-dev.txt`, `npm-audit-omit-dev.json`	Install log and `npm audit --omit=dev`
`npm-build.txt`, `npm-lint.txt`, `npm-test-ci.txt`	`npm run build` (exit 0), `npm run lint` (exit 0), `npm run test:ci` (27 suites, 551 tests, 97.57% statements)
`lighthouse-root.json`, `lighthouse-dashboard-create.json`, `lighthouse-about.json`, `lighthouse-summary.txt`	Lighthouse 12.8.2 reports
`onchain.txt`	Read-only `cast call` / `cast codesize` / `cast code` results for every address in 2.2 and the role holders
`source-verification.txt`	Sourcify, Blockscout and Etherscan checks; bytecode comparison against the audited build
`abi-diff.txt`	Frontend `ChainvoiceABI.js` vs `forge inspect Chainvoice abi`
`live-site.txt`	Live response headers and script tags
`secret-scan.txt`	Secret-pattern scan of the built bundle
`chrome-test-prompt.md`, `chrome-test-prompt-part2.md`, `browser-pass-2026-10-24.pdf`, `browser-pass-2026-10-24-part2.pdf`, `onchain-test-invoices.txt`	Functional test scripts, the two browser test result reports, and the on-chain state of the test invoices
`poc/AuditPoC.t.sol`, `poc/forge-poc-output.txt`	CV-SC-01 PoC (kept outside the clone; copy into `contracts/test/` to run)
`poc/fe-payload-poc-1.mjs`, `poc/fe-payload-poc-2.mjs` and outputs	CV-FE-01 PoCs (run from Chainvoice/frontend; they import the real modules by absolute file:///R:/… path, so adjust the base path on another machine)

Key excerpts:

```
forge coverage (src/Chainvoice.sol): 98.93% lines (185/187), 93.10% statements (243/261),
                                     70.49% branches (43/61), 100.00% funcs (23/23)
slither: 102 detectors, 11 results: arbitrary-send-eth (1), reentrancy-balance (1), calls-loop (3),
         cyclomatic-complexity (1), solc-version (1), low-level-calls (4)
npm audit --omit=dev: 31 vulnerabilities (18 moderate, 13 high), 0 critical
```

Appendix B. Glossary

Term	Meaning
EOA	Externally owned account: a wallet controlled by a private key
CEI	Checks-effects-interactions: update state before calling other contracts
PoC	Proof of concept: minimal steps or code that demonstrate an issue
Payload	The full invoice details (parties, items, dates) that travel off-chain; only its hash is on-chain
invoiceDataHash	`keccak256` of the payload serialised with sorted keys, stored in the contract as a commitment
ECIES	Elliptic Curve Integrated Encryption Scheme: encrypts a payload to a recipient's secp256k1 public key
ThruBox relay	External mailbox server that stores encrypted payloads for recipients to poll
devFee	Flat native-coin fee per paid invoice, collected in the contract and withdrawn to `devFeeReceiver`
EIP-7702 delegation	An EOA whose code is set to `0xef0100 ‖ address`, delegating execution to a smart-account contract
Share link / `cv1.` token	A URL fragment carrying a compressed invoice payload that another user can import and verify against the chain